Stateless Architecture

A **stateless architecture** is a system design where the server does **not store client session data locally** between requests. Every request is treated as an independent request.

Instead of keeping user state inside application servers, all shared data is stored in external storage like:

```
None
Database

Redis Cache

Session Store

Object Storage
```

1. How It Works

In a stateless system:

```
None
User Request → Load Balancer → Any Available Server

                        ↓

              Server fetches state from shared store
```

So any server can handle any request.

Example:

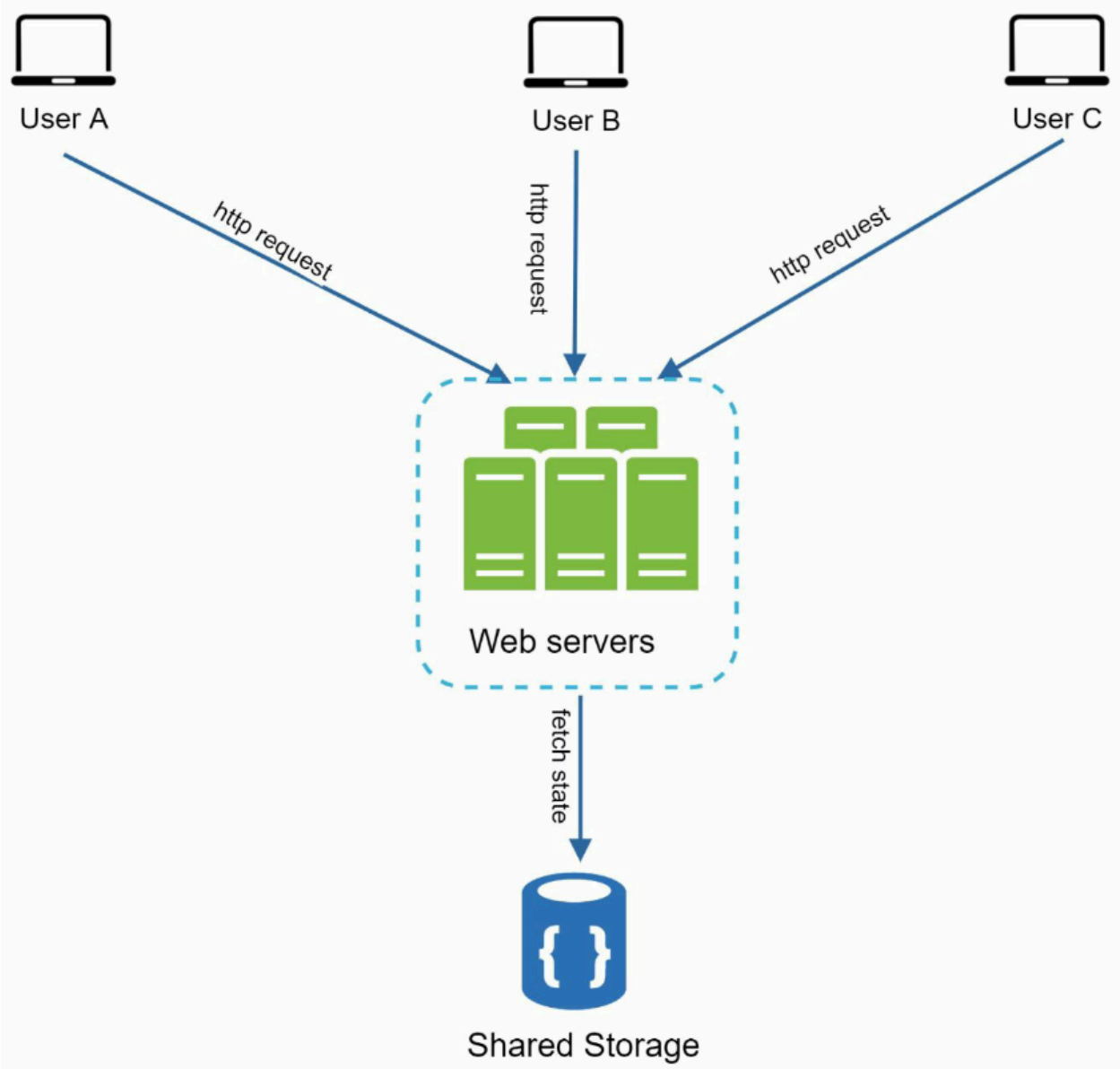
```
Login Request → Server 1

Next API Call → Server 3

Next Request → Server 2
```

All work correctly because session/state is stored centrally.

2. Example



Instead of storing User A session in one server:

```
None
Server 1:

User A Session
```

Store it in shared Redis:

```
None
Redis:

User A Session

User B Session

User C Session
```

Now requests can go to any server.

3. Benefits of Stateless Architecture

A. Easy Horizontal Scaling

Add more servers anytime:

```
None
3 servers → 10 servers
```

Load balancer distributes traffic automatically.

B. High Availability

If one server crashes:

```
None
Server 2 Down
```

Requests go to other servers.

No user session lost (if stored externally).

C. Simpler Load Balancing

No need for:

```
None
Sticky Sessions

Session Affinity
```

Any request can go anywhere.

D. Faster Deployments

Replace servers one by one without breaking active users.

E. Better Auto Scaling

Cloud platforms can add/remove servers dynamically.

4. Typical Shared Stores

Session Storage

```
None
Redis
Memcached
Database
```

User Data

```
None
MySQL
PostgreSQL
MongoDB
```

Files

```
None

S3

Blob Storage

CDN
```

5. Example Login Flow

```
None

User logs in

→ Server validates credentials

→ Session token stored in Redis

→ Token returned to client
```

Next request:

```
None
Any server receives token

→ Reads session from Redis

→ Authenticated
```

6. Real World Systems Using Stateless Design

Most modern systems use stateless application servers:

```
Netflix

Amazon

Uber

Airbnb

Google APIs
```

7. Drawbacks

A. Extra Network Calls

Need to fetch session/state externally:

```
Server → Redis / DB
```

Adds slight latency.

B. Shared Store Becomes Critical

If Redis/session DB fails:

```
None
Login/auth may fail
```

Need replication + HA.

C. More Infrastructure

Need:

```
None
Redis Cluster

DB Replicas

Monitoring

Failover
```

8. Best Practice Design

```
None
Client
 ↓
Load Balancer
 ↓
Stateless App Servers
 ↓
Redis (sessions/cache)
 ↓
Database
```

9. Stateful vs Stateless

Feature	Stateful	Stateless
Session stored in server	Yes	No
Any server handles request	No	Yes
Easy scaling	Hard	Easy

Failure recovery	Hard	Easy
Sticky sessions needed	Yes	No

10. Interview Answer

If asked scalable web architecture:

```
None
Use stateless app servers behind load balancer.

Store sessions in Redis.

Store persistent data in DB.

Auto-scale servers horizontally.
```

Strong industry-standard answer.

11. One-Line Summary

Stateless architecture stores client state in shared external storage instead of application servers, allowing any server to handle any request, which makes the system scalable, reliable, and cloud-friendly.